Tools & Files

Tools are how your agent does things beyond chatting — run code, read a file the user attached, query a database, call an API. The SDK manages their whole lifecycle (creation, cleanup, session scoping) for you.

You declare tools in your agent's `tools=[...]` list. This page covers the tool types, working with files, the built-in code sandbox, and the database connector.

Tool types

Simple tool

A plain function the model can call. The factory just returns the tool:

```
from ai_foundry_platform_sdk.agent import ToolDefinition
from strands_tools.calculator import calculator

ToolDefinition(
    name="calculator",
    factory=lambda session_id, metadata: calculator,
)
```

MCP tool (connect to an external tool server)

MCP is a standard protocol for tool servers. Use `mcp_tool(...)` to connect to any MCP server; the SDK opens the connection, discovers the available tools, and cleans up afterwards. You choose how it authenticates with the `auth=` argument:

```
from ai_foundry_platform_sdk.tools import (
    mcp_tool, NoAuth, StaticBearer, IdentityOAuth2, IdentityApiKey,
)
```

```
# Internal server, no auth header
mcp_tool(name="local", url="http://internal-mcp:8080", auth=NoAuth())

# Managed OAuth2 (on-behalf-of the calling user when a token is present, app-to-app otherwise)
mcp_tool(
    name="msgraph",
    url="https://<id>.gateway.bedrock-agentcore.eu-central-1.amazonaws.com/mcp",
    auth=IdentityOAuth2(provider="msgraph-obo", scopes=["User.Read"]),
)

# Static API key managed by the platform identity service
mcp_tool(name="vendor", url="https://vendor-mcp/mcp",
auth=IdentityApiKey(provider="vendor-api-key"))
```

Choosing the OAuth2 flow. `IdentityOAuth2` accepts a `flow` argument to pick how the token is obtained:

`flow`	Use it for
`"on_behalf_of"`	Acting **as the calling user** from a deployed (headless) agent — the right choice for AgentCore Gateway calls. The user's workload token is captured for you; no extra wiring.
`"user_federation"`	Interactive user sign-in + consent (cannot complete headlessly).
`"m2m"`	App-to-app (service credentials, no user).

Omit `flow` to keep the default (user federation when a caller token is present, app-to-app otherwise). An invalid value fails fast when the agent starts, not on the first request.

If an MCP server is unreachable or returns no tools, the SDK logs a warning and skips that tool — your agent still starts. Tool discovery is fully paginated, so a gateway exposing many tools surfaces all of them, not just the first page. (AgentCore Gateway endpoints are just MCP servers reached this way; there's no separate helper.)

Multiple gateways initialise in parallel. When an agent wires several MCP gateways, the SDK sets them up concurrently, so startup time (and time-to-first-token) doesn't grow linearly with the number of gateways. Tool order is preserved regardless. The parallelism is capped (default

8) and tunable per agent with the `AGENT_MCP_INIT_MAX_WORKERS` environment variable — lower it if a large fleet is exchanging too many identity tokens at once.

Two search-enabled gateways no longer collide. AgentCore Gateway exposes a built-in semantic-search tool under a fixed name; wiring two or more such gateways used to clash on that shared name and could fail the build. The SDK now gives each gateway's search tool a gateway-scoped, agent-facing name (`<gateway>___...`) while routing still reaches the correct gateway — so you can safely enable search on multiple gateways at once.

Session-bound tool

Some tools need a session that starts, gets used, and is cleaned up (the code sandbox is the main example). Set `requires_session=True` and the SDK manages start and cleanup for you — at request end it calls `stop()`, `close()`, or `__exit__()` on the resource if present.

```
ToolDefinition(
    name="my_session_tool",
    factory=my_factory,       # returns an object with a stop()/close() method
    requires_session=True,
    tool_attr="run",          # the callable attribute on that object
)
```

Conditional tool

Return `None` from the factory to skip a tool for a particular request — useful for feature flags per caller:

```
def advanced_factory(session_id, metadata):
    return advanced_tool if metadata.get("enable_advanced") else None

ToolDefinition(name="advanced", factory=advanced_factory)
```

Passing the user's identity to a tool

The caller's login token is available to your tool factory, so a tool can act **as the user** against a downstream system (Microsoft Graph, Databricks, SAP, ...):

```
def my_tool_factory(session_id, metadata):
    token = metadata.get("caller_token")
    return MyClient(url="https://graph.microsoft.com", token=token)
```

This is end-to-end identity propagation — the original user's token flows from the frontend, through the agent, to the tool. More in Security & Access.

Files the user attaches

When a user sends a file along with their message, your agent can work with it — **without** you writing any upload handling. The SDK detects attachments (whether sent as inline bytes or as a link) and makes them available to file-aware tools. Nothing is downloaded until the model actually asks for the file.

There are two ways an agent can consume a file, and you can enable either or both:

If you want the…	Use	The model calls…
Model to read/understand the file (classify an image, answer questions about a CSV)	`files_tool()`	`read_file(name)` — pulls the content into the model's context
Code sandbox to process the file (`pandas.read_csv`, open a spreadsheet)	`code_interpreter_tool(...)`	`load_files()` — stages the file into the sandbox

```
from ai_foundry_platform_sdk.tools.files import files_tool

agent = AgentDefinition(
    tools=[files_tool()],   # optional: files_tool(max_read_bytes=4 * 1024 * 1024)
    ...
)
```

Supported for direct reading (`read_file`): images (`png`, `jpeg`, `gif`, `webp`) and text formats (`txt`, `csv`, `md`, `html`). **PDF and Office documents (`docx`, `xlsx`, ...) are *not* supported for direct reading** — route those through the code sandbox's `load_files()` instead. Files above the size cap (default 4 MB) return a clear error to the model rather than failing silently.

Code sandbox (`execute_python`)

The SDK wraps the AWS **Code Interpreter** as a session-bound tool. The model gets an `execute_python` tool that runs Python in a secure, isolated sandbox — ideal for data analysis, charts, and file transformation that the model shouldn't attempt in its head.

```
from ai_foundry_platform_sdk.tools.code_interpreter import code_interpreter_tool

agent = AgentDefinition(
    tools=[
        code_interpreter_tool(
            region="eu-central-1",
            session_timeout_seconds=900,   # sandbox lifetime (default 15 min)
        ),
    ],
    ...
)
```

Requires the `code-interpreter` extra (`pip install ai-foundry-platform-sdk[code-interpreter]`; included in `[all]`).

How it behaves:

- The sandbox starts **lazily** — only when the model first runs code. A conversation where it never runs code never creates (or pays for) a sandbox.
- Sandbox state persists **across calls within one request**, but not between requests. Each new request starts fresh (re-attach files each turn if needed).
- **Deliverables:** tell the model to save user-facing files (charts, reports) under `outputs/` using relative paths like `outputs/chart.png`. At the end of the run the SDK collects everything under `outputs/` and delivers it to the caller automatically (attached for A2A,

emitted as events for AG-UI). Files written elsewhere (e.g. `/tmp/`) are not delivered. It's worth repeating this rule in your domain prompt.

Large files (S3)

Inline delivery is capped (10 MB on A2A, 5 MB on AG-UI). For bigger files — common in data analysis — the sandbox can route files through an S3 bucket with time-limited download links instead, with no inline size cap. You enable it by giving the tool a bucket:

```
code_interpreter_tool(
    region="eu-central-1",
    s3_bucket="my-agent-files-dev",   # opt-in: enables the S3 path
    s3_prefix="agent-outputs",        # optional
)
```

The bucket must already exist and the agent's runtime role must be granted access to it (see Deployment). The SDK never handles AWS keys — access is granted through the runtime's role.

Database connectors

For direct database access (when you have credentials and an endpoint), the SDK ships built-in connectors. Each gives the model two tools: `get_schema` (discover tables and columns) and `execute_sql` (run **read-only** queries).

```
from ai_foundry_platform_sdk.tools.postgres import create_postgres_tools

agent = AgentDefinition(
    tools=[
        create_postgres_tools(
            host=os.environ["POSTGRES_HOST"],
            database="analytics",
            schema="reports",
            credentials_secret="my-postgres-creds",   # AWS Secrets Manager name
            max_rows=1000,
        ),
    ],
```

```
    ...
)
```

Install with `pip install ai-foundry-platform-sdk[postgres]` (or `[connectors]` for all of them).

Built-in safety on every connector:

- **Read-only guard** — only `SELECT` / `SHOW` / `DESCRIBE` / `EXPLAIN` / `WITH` are allowed. Anything that could modify data is rejected before it reaches the database. (This is intentional and not configurable in v1.)
- **Friendly output** — results render as Markdown tables, with a notice when truncated to `max_rows`.
- **Safe errors** — failures come back as readable messages to the model, never as crashes.

> **Databricks** is intentionally not a built-in connector — connect it through an MCP/Gateway endpoint instead (see MCP tool).

Where next

- Building Your Agent — the definition that holds these tools.
- Security & Access — passing the user's identity through to tools.
- Deployment — granting the runtime access to S3 or other AWS resources.
- Reference — tool settings and environment variables.